Exercise 9: Working with Regular Expressions

1. Given the following list of potential e-mail addresses:

["HumphreysDJ@cardiff.ac.uk", "support@python.org", "support.python@org", "WilliamsPT@programs.com", "AliyevOR@information.gr", "SatoUE@aberdare.ac.uk", "myemail.edu"]

Use regular expressions to write separate functions which can do the following:

(a) Detect whether an e-mail address is "valid" (simply, it must contain two parts separated by an "@" symbol, and at least one suffix following a period "."),

i.e. <username>@<domain>.<tld>

(b) Extract the username portion of each e-mail address, and return it;

(c) If the username portion has the form <Surname><Initial 1><Initial 2> (e.g. "HumphreysDJ"), extract the surname and both initials, and return them in a list;

(d) Detect whether the address ends in an educational suffix (".edu" or ".ac.uk").

Add code which calls your functions on each e-mail address, and prints out the information returned by them (validity, username, surname and intiials is available, and whether the address is an educational one.

Download the archive "articles.zip" from Learning Central (from the "CMT307/Python Bootcamp/Exercises" folder), unzip it on your computer, and upload all the text articles files ("article1.txt" to "article5.txt") to your Google Colab (temporary) file area.

2. Given these articles, write code which does the following for each article

(a) Loads in the article's text;

(b) Uses a regular expression to extract all the words which begin with capital letters, after a whitespace (e.g. " Cardiff" should match, but not "Travel");

(c) Uses a regular expression to extract all the numbers which follow a whitespace, and all monetary values / prices which follow a whitespace (e.g. "10,000", "$2.54", "£1.43", "2024" etc.);

(d) Prints out the list of all matches for each article (perhaps with a count of the number of occurrences of each string found; you might try the Counter's method ".most_common()" and slicing to fetch only the top 10 word counts, or the top 20).

From looking at the returned strings, can you tell:

(f) What the general topic of each article is?

(g) What subject _all_ the articles come from?

Is it possible to use a technique such as this to understand the contents of articles more quickly than reading them individually?